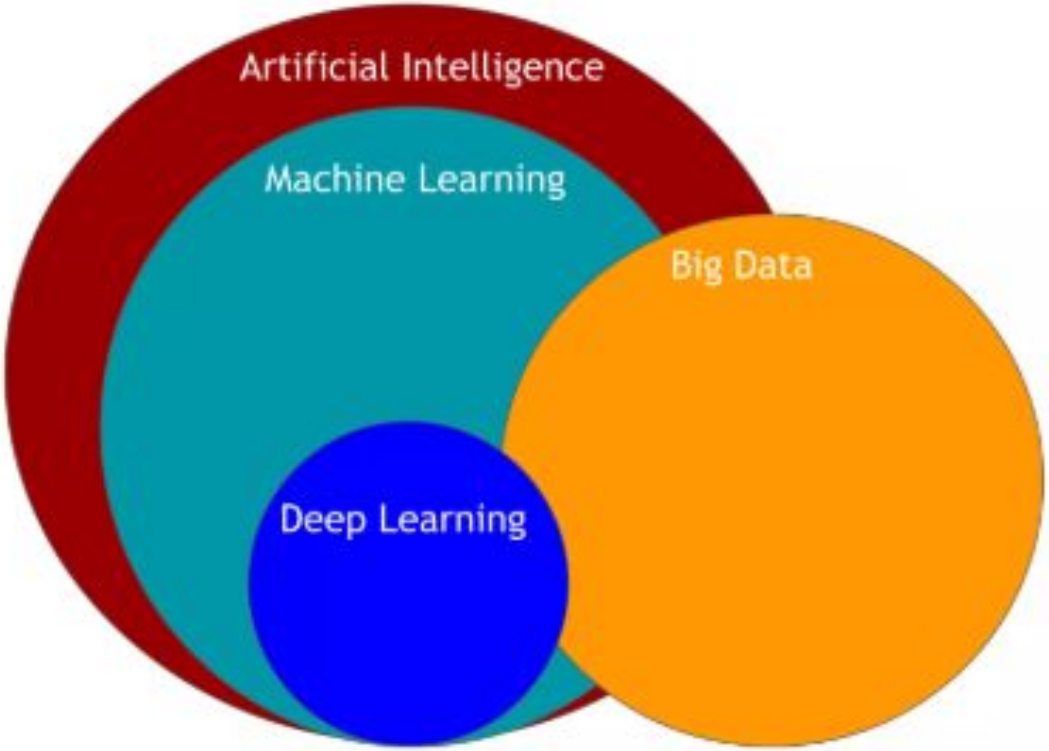
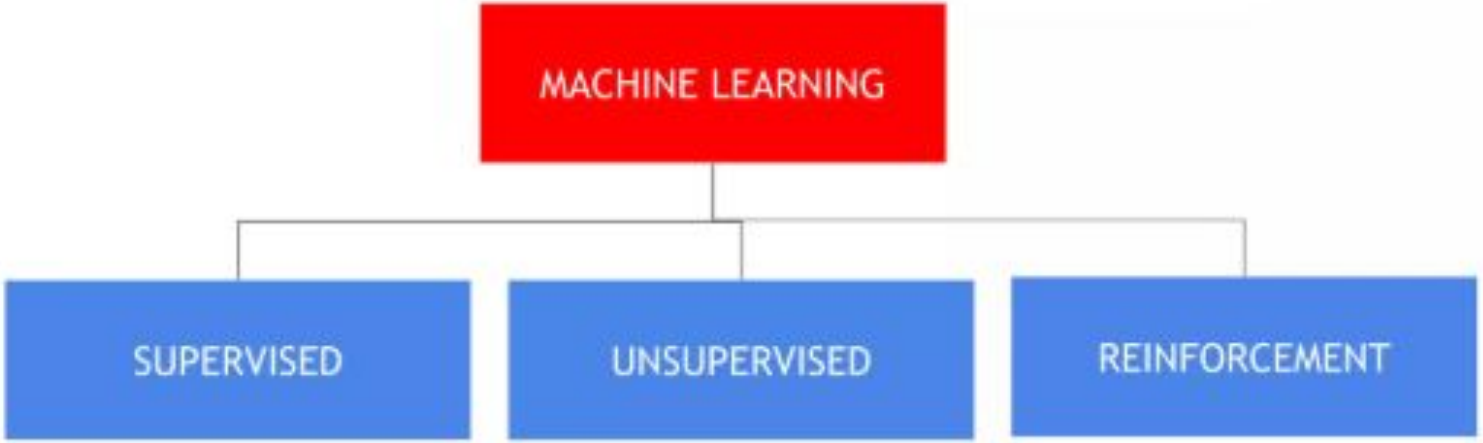
Aula 5 - Usando o pacote YOLO

Davi Duarte de Paula
davi.duarte@unesp.br

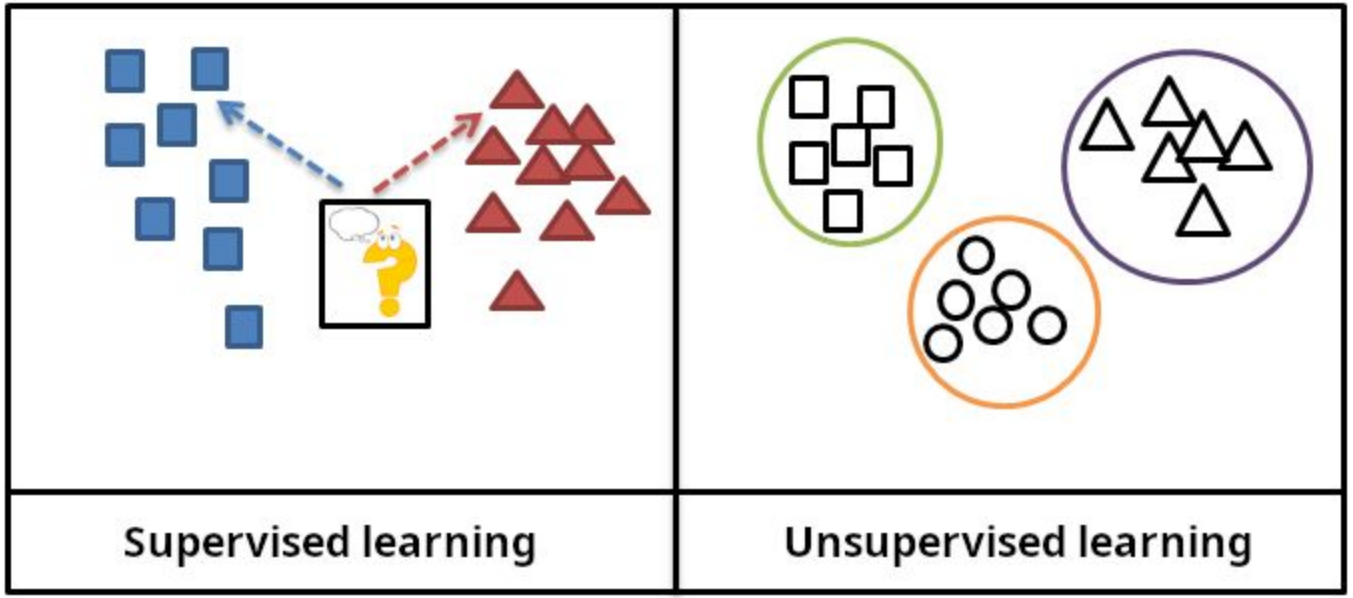
Subáreas da Inteligência Artificial



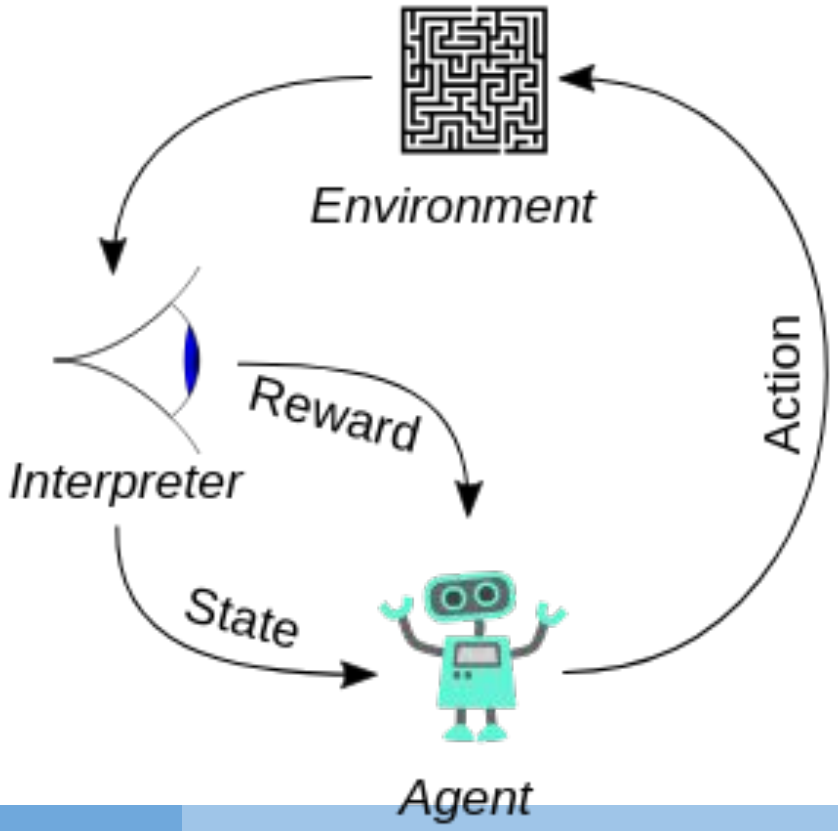
Subáreas de Aprendizado de Máquina



Aprendizado de Máquina

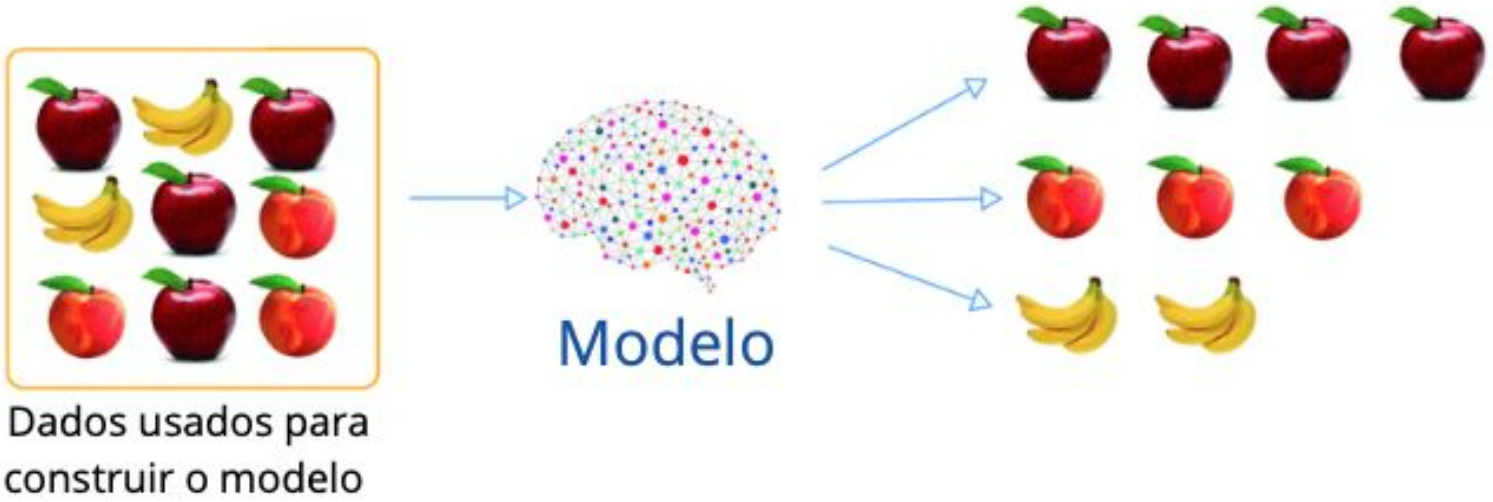


Aprendizado de Máquina



Aprendizado Supervisionado

DADOS ROTULADOS!

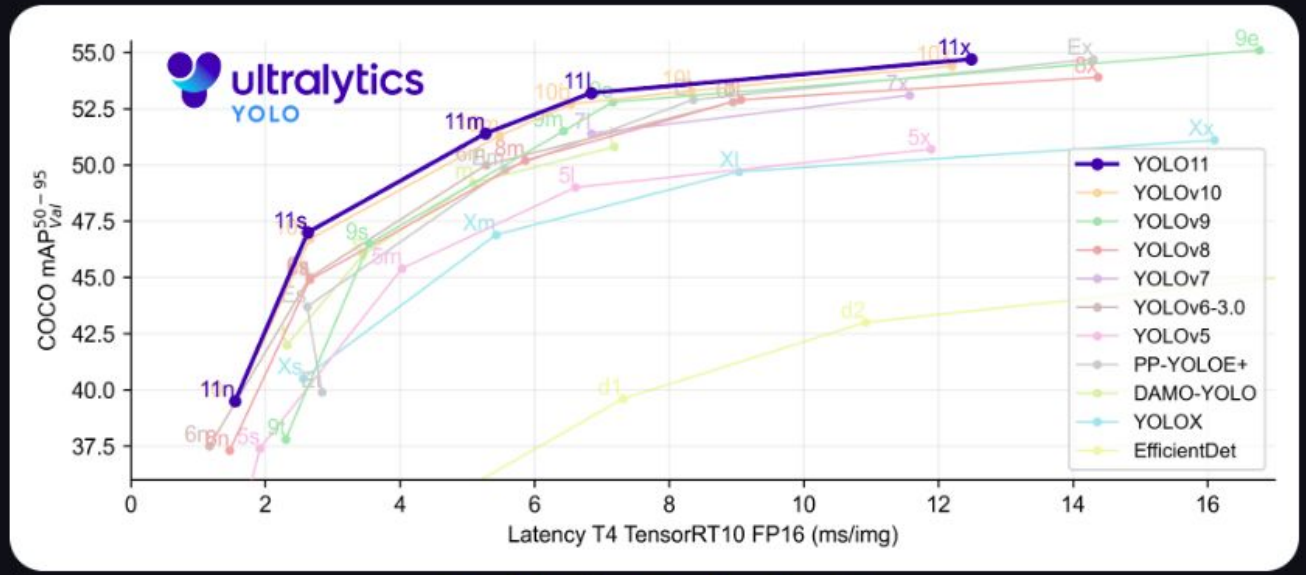


YOLO

README | Code of conduct | Contributing | AGPL-3.0 license | Security

Find detailed documentation in the Ultralytics Docs. Get support via GitHub Issues. Join discussions on Discord, Reddit, and the Ultralytics Community Forums!

Request an Enterprise License for commercial use at Ultralytics Licensing.



Documentation

Contributors 472

+ 458 contributors

Deployments 322

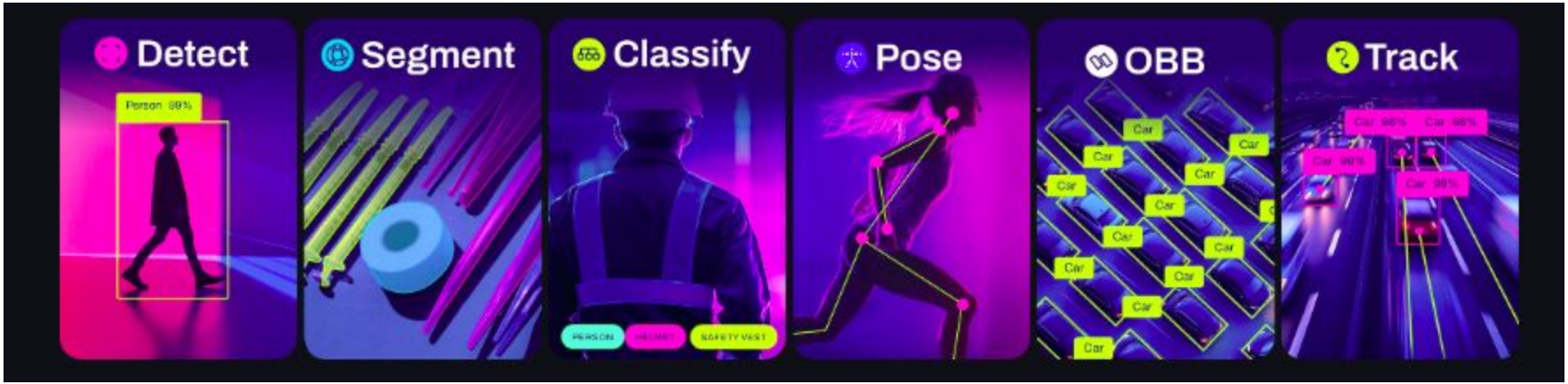
- **Release - PyPI** yesterday
- **github-pages** 2 years ago

+ 320 deployments

Languages

- **Python** 99.5%
- **Other** 0.5%

- Modelo baseado em CNN que possui várias funcionalidades



Exemplo 1

- Instale o pacote ultralytics:
  - pip3 install ultralytics
- Execute o exemplo1.py e exemplo2.py

Formato dos dados

- A Yolo define as caixas delimitadoras em relação ao tamanho da imagem.
  - classe x_center y_center width height
- Cada caixa é composta pela classe, pelas coordenadas centrais e pela largura e altura

Caixas delimitadoras de exemplo

- O arquivo a seguir mostra 3 caixas delimitadoras, a primeira possui classe 0 (pessoa), sendo o ponto central em 0.5 e 0.5 (relativo à imagem), com largura e altura de 0.3 e 0.4

0 0.5 0.5 0.3 0.4

1 0.3 0.6 0.2 0.2

1 0.3 0.34 0.5 0.6

- Cada imagem possui um arquivo de rótulo. Aquelas que não possuírem significa que elas não tem nenhum objeto.

Atividade

- A partir do código exemplo3.py, implemente uma verificação de colisão entre carros (classe 3)

Treinar a YOLO

- É possível treinar a YOLO em um conjunto de dados personalizado para detectar qualquer objeto.
-

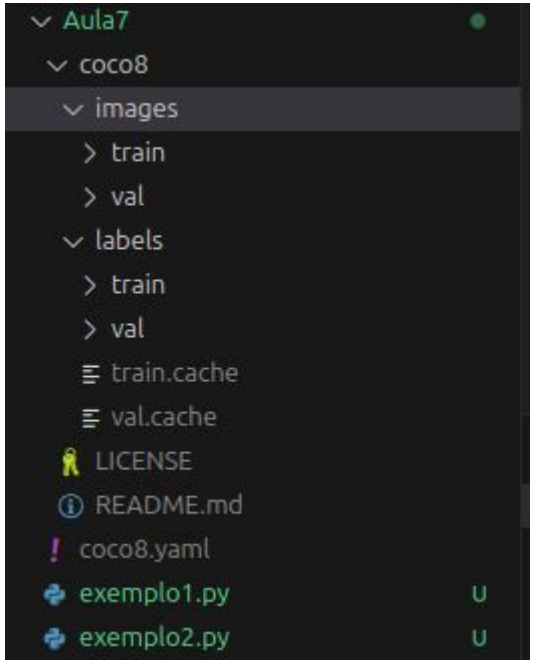
Treinar a YOLO

- Vamos usar o conjunto de dados COCO8, uma versão simplificada do dataset COCO, contendo detecção de objetos para 80 classes, como relógio, livro, laptop,etc.
- O conjunto de dados é configurado pelo arquivo coco.yaml
-

```yaml
path: coco8 # dataset root dir
train: images/train # train images (relative to 'path') 4 images
val: images/val # val images (relative to 'path') 4 images
test: # test images (optional)

# Classes
names:
  0: person
  1: bicycle
  2: car
  3: motorcycle
  4: airplane
  5: bus
  6: train
  7: truck
```

Treinar a YOLO

- Faça o download do conjunto de dados COCO8.
- Organize os arquivos como a seguir:
-



- Crie um arquivo em Python para treinar o seu detector de objetos

```python
from ultralytics import YOLO
import os

# Load a model
model = YOLO("yolo26n.pt")  # load a pretrained model (recommended for training)

# Train the model
results = model.train(data="coco8.yaml",
                      epochs=100,
                      imgsz=640,
                      project="/media/davi/Aula7/runs",
                      name="experiment1")
```

- Execute o arquivo e veja as métricas
- Cada seção é uma época.

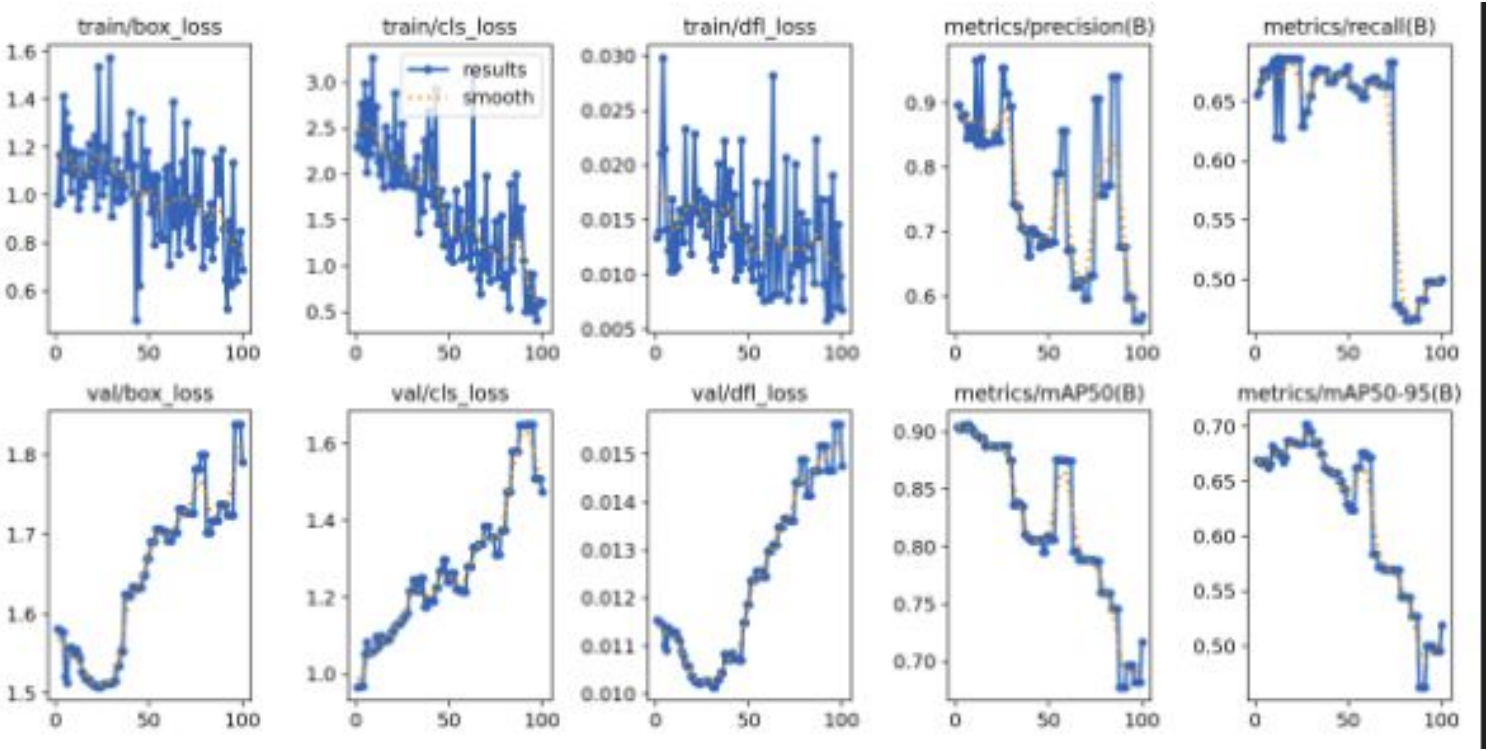
```
TERMINAL

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      1/100         0G     0.9634      2.291    0.01334         30        640: 0% ──────────
      1/100         0G     0.9634      2.291    0.01334         30        640: 100% ━━━━━━━━━━━━
4.1s/it 4.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
━━━━━━━━ 1/1 1.3s/it 1.3s
                   all          4         17      0.897      0.656      0.904      0.669

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      2/100         0G      1.169      2.443    0.01394         30        640: 0% ──────────
      2/100         0G      1.169      2.443    0.01394         30        640: 100% ━━━━━━━━━━━━
3.3s/it 3.3s
```

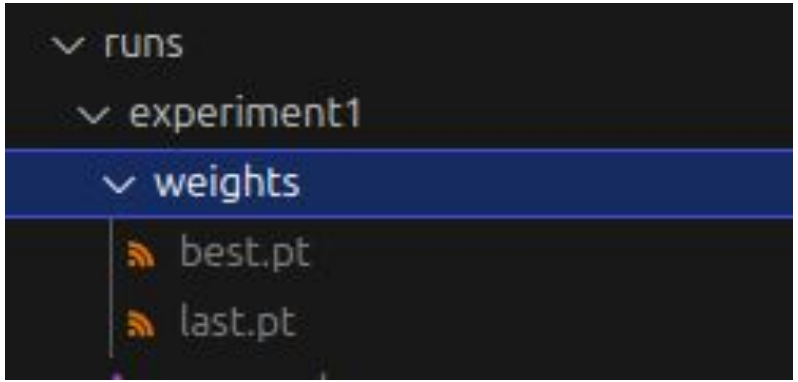
Resultados

- Os gráficos e o modelo treinado estão armazenados na pasta **runs/experiment1** e **runs/experiment1/weights**, respectivamente.
-

Resultados

- Results.png





- best.pt é o melhor checkpoint encontrado
- last.pt é checkpoint da última época (pode não ser o melhor)

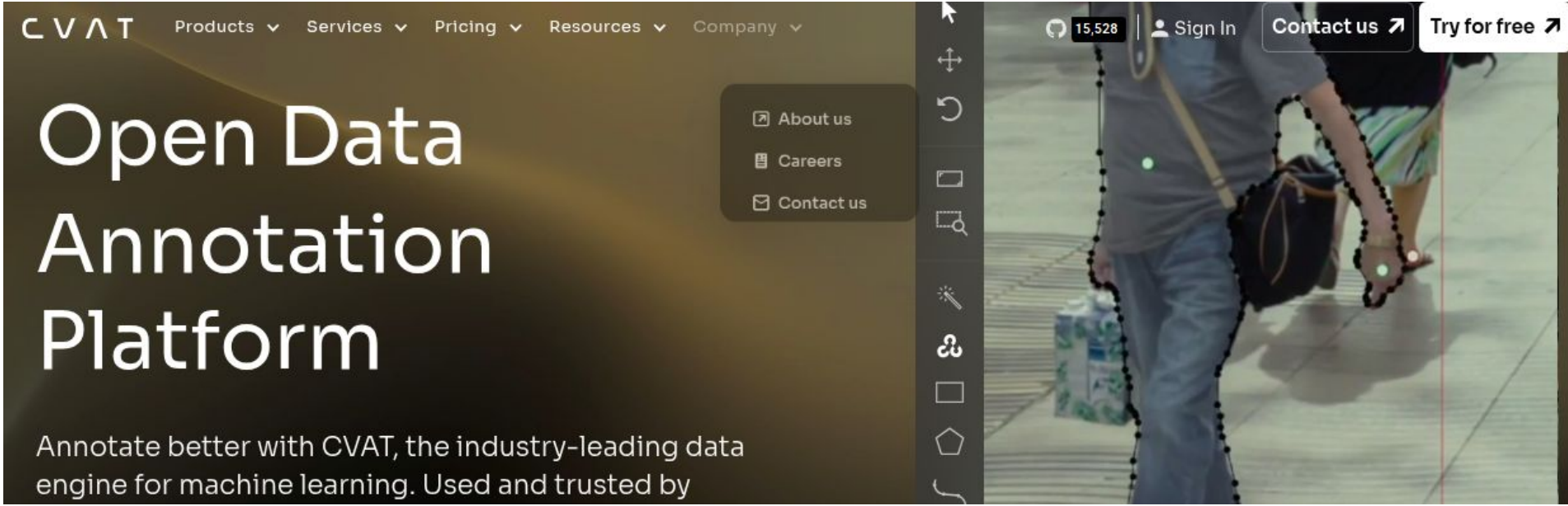
Executando o seu checkpoint

- No exemplo 1, altere o PATH do modelo carregado para o **last.pt**

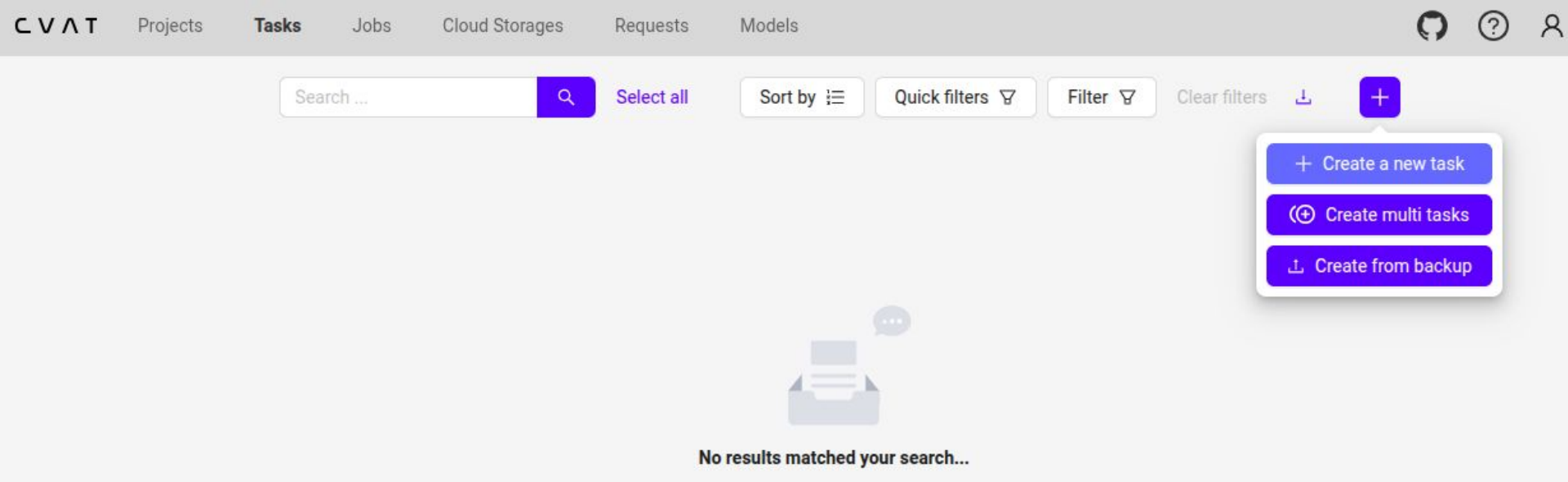
```python
from ultralytics import YOLO

# Load a pretrained YOLO26n model
model = YOLO("last.pt")

# Train the model on the COCO8 dataset for 100 epochs
train_results = model.train(
    data="coco8.yaml",  # Path to dataset configuration file
    epochs=100,  # Number of training epochs
    imgsz=640,  # Image size for training
    device="cpu",  # Device to run on (e.g., 'cpu', 0, [0,1,2,3])
)
```

Rotulando Dados

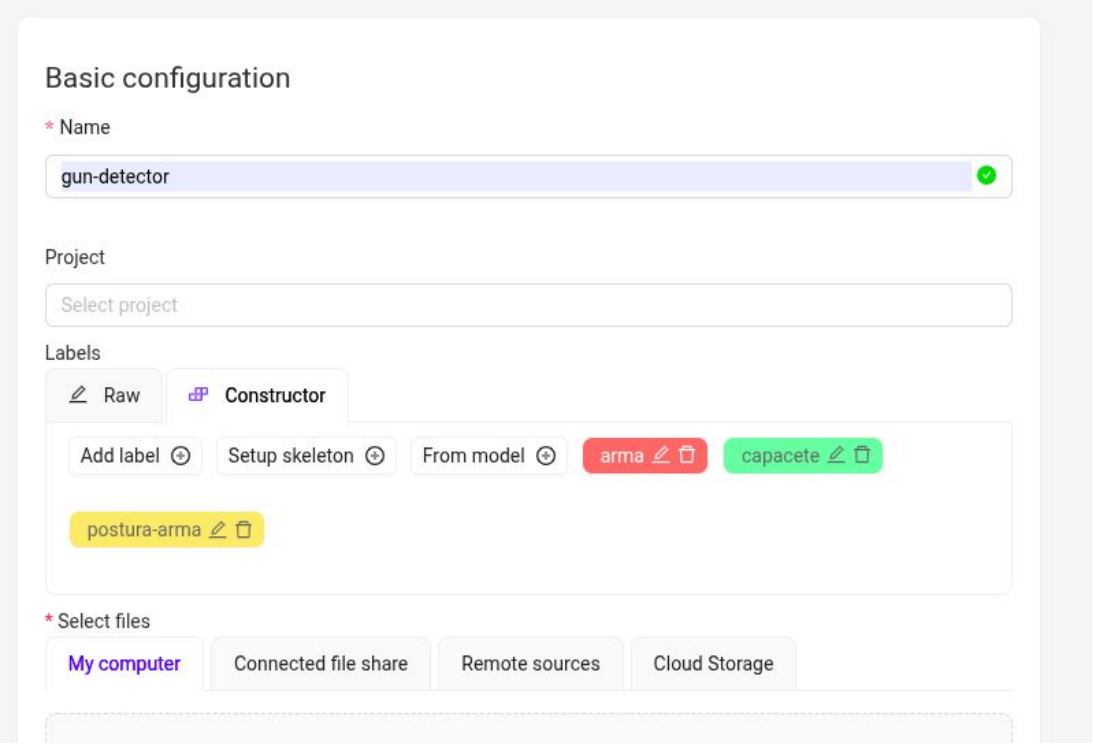
- A rotulagem de dados é a etapa muito custosa e deve ser preparada com cuidado.
- Amostras rotuladas incorretamente podem afetar a qualidade do modelo.
- Vamos usar a plataforma cvat.ai. Faça uma conta gratuita.



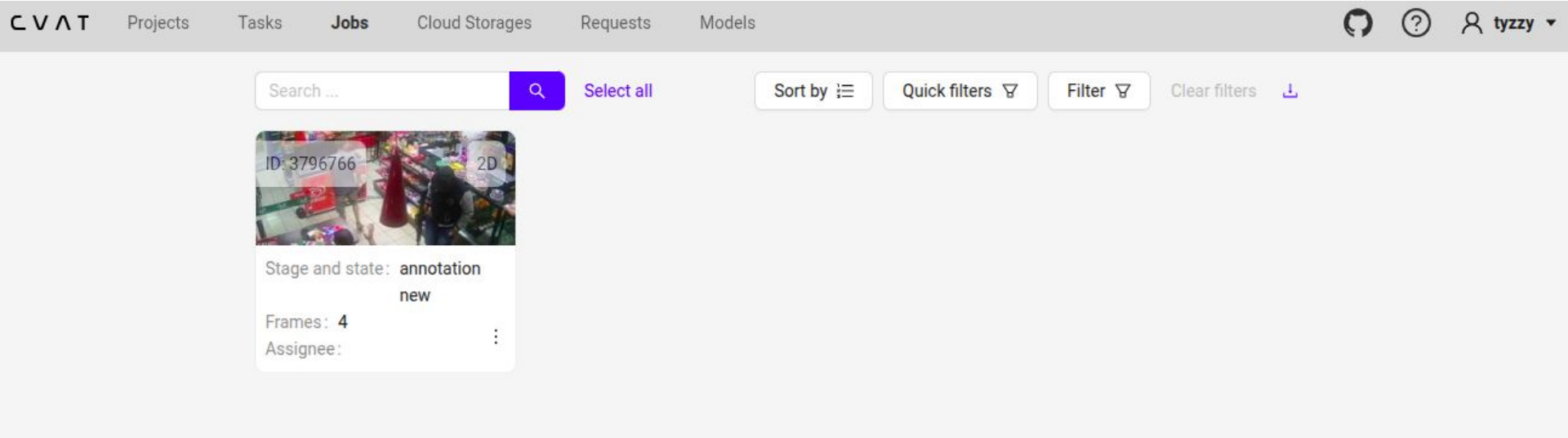
- Clique em “Create new task”



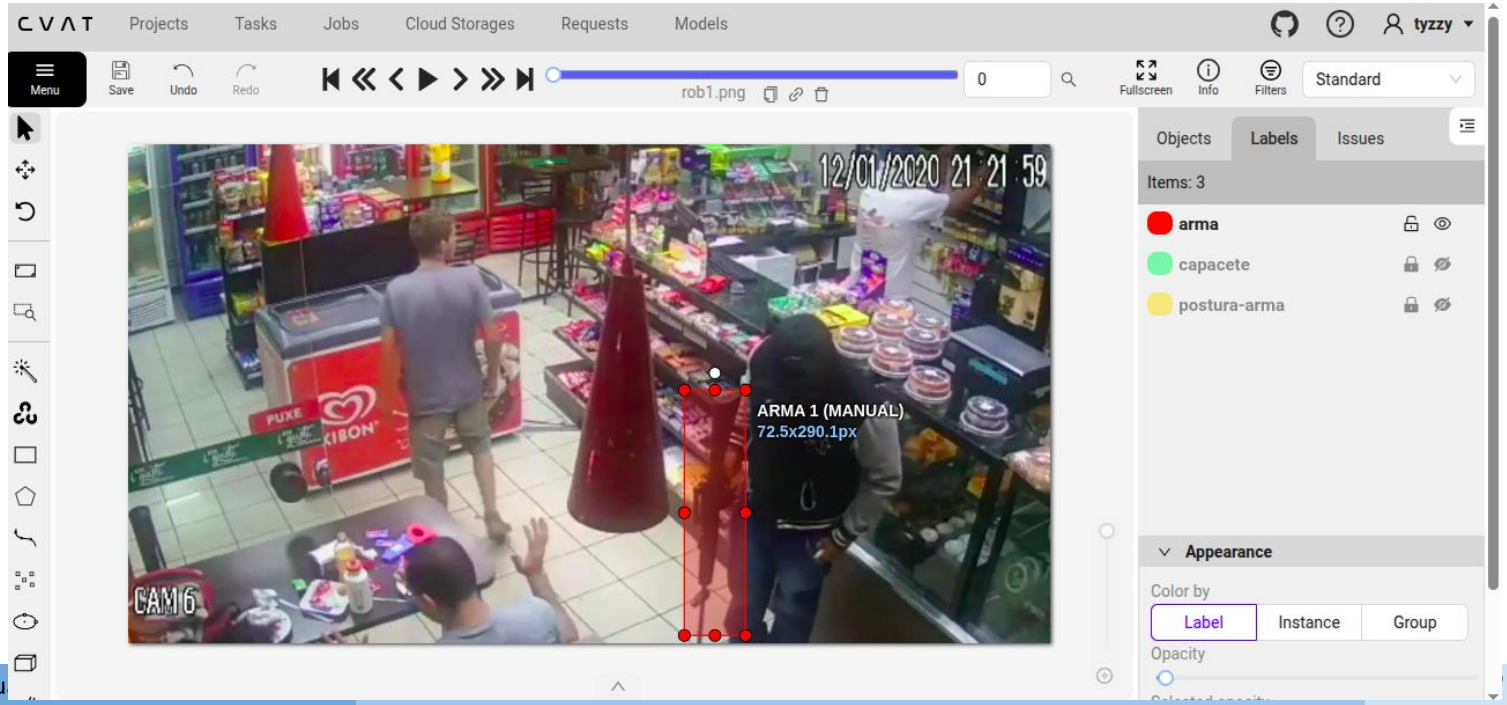
- Insira um nome para a tarefa de rotulagem, escolha os rótulos (o que você vai detectar) e envie suas imagens



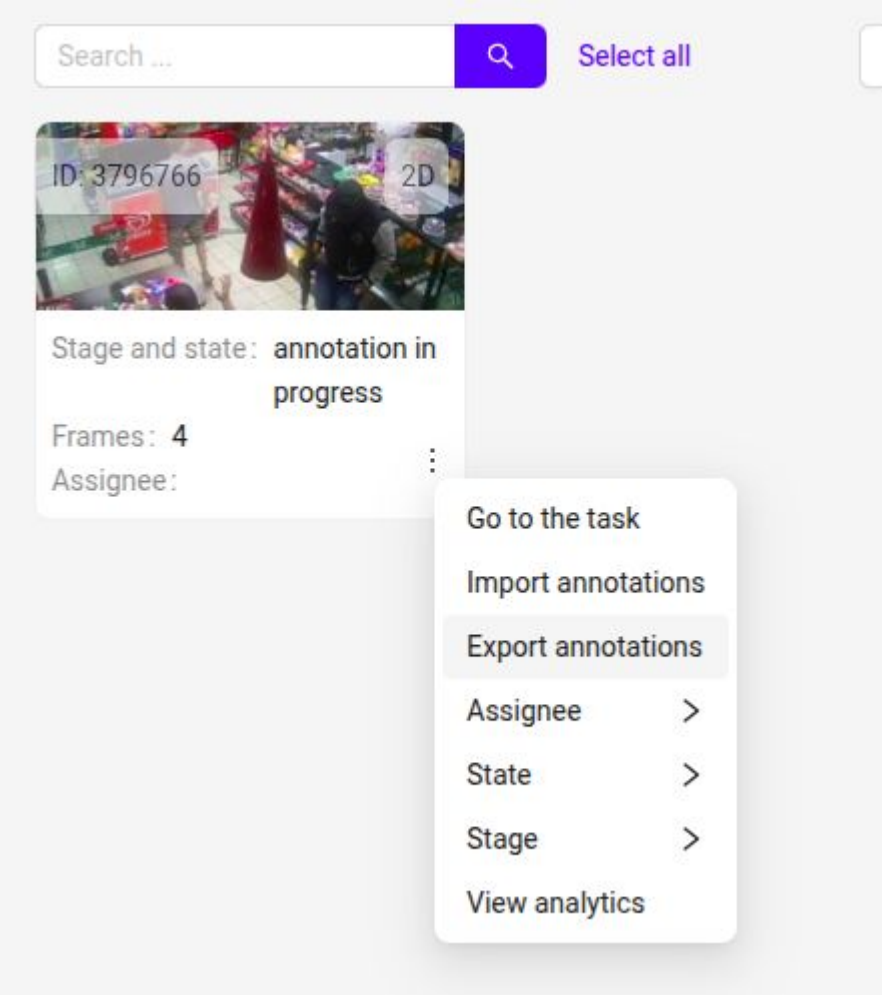
- Na guia JOBS, selecione o trabalho da tarefa que você acabou de criar



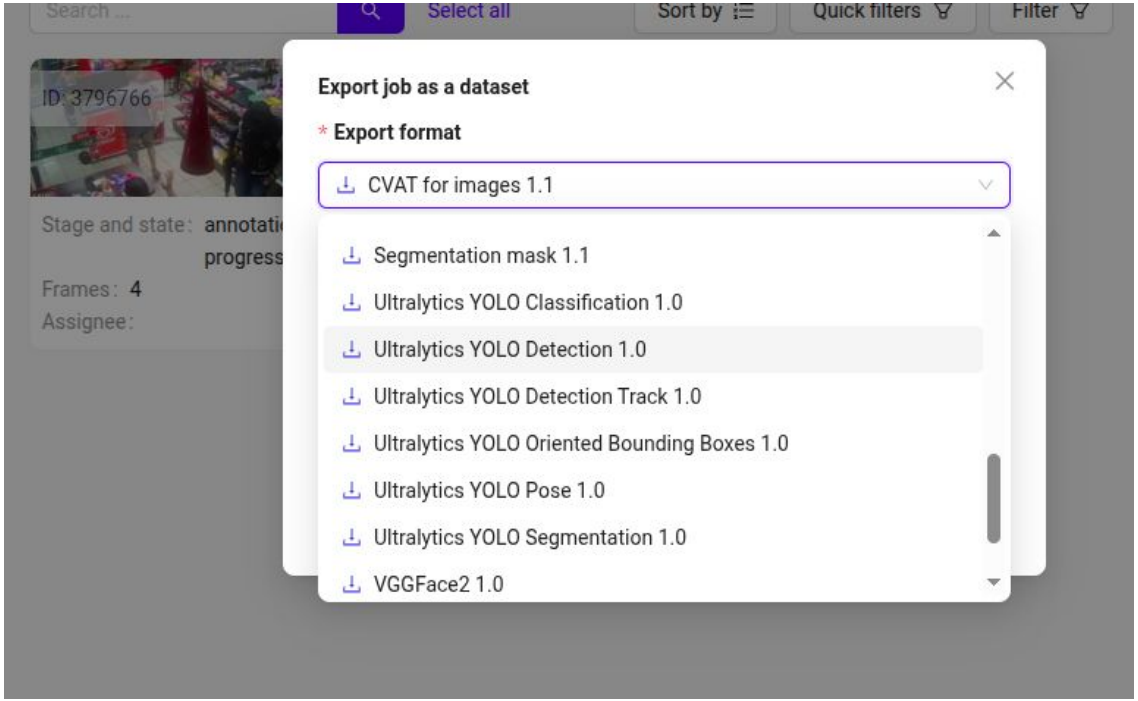
- Na janela que aparecer, passe por todas as imagens e marque com a classe correta todos os objetos que aparecerem



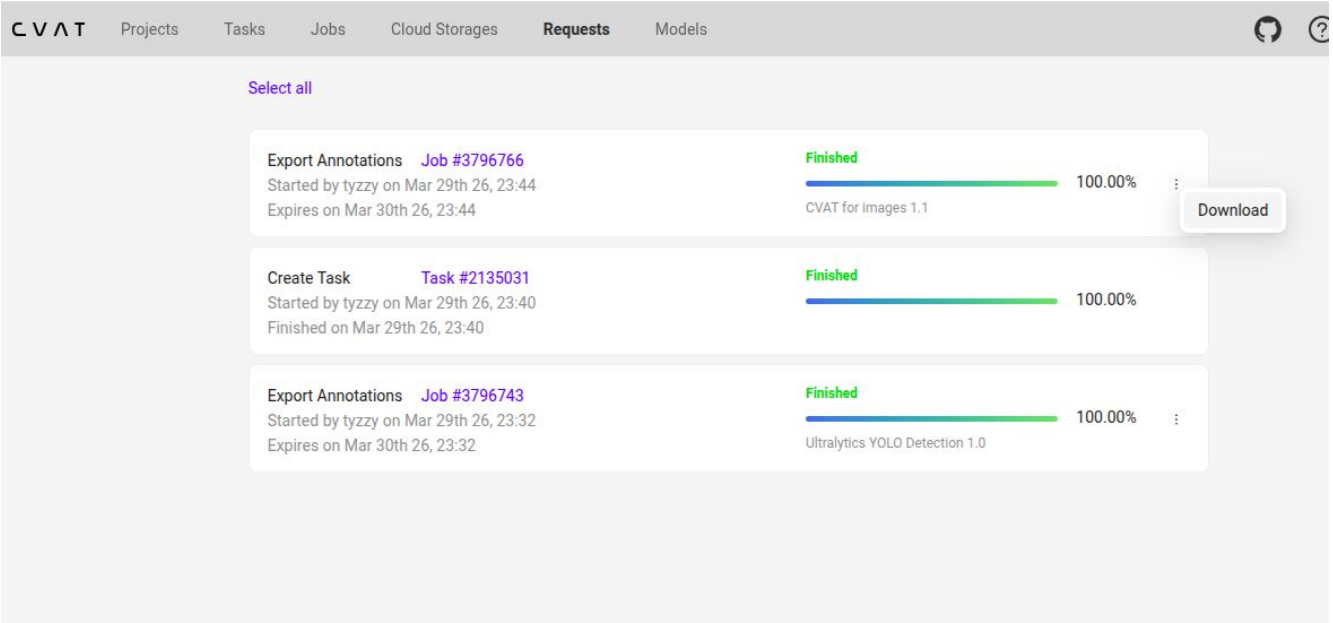
- Quando você terminar de rotular todas as imagens, vá em JOB, 3 pontinhos, Exporte Annotation



- No formato de exportação, seleciona Ultralytics Yolo Detection



- Em requests, faça o download dos rótulos para você usar no treinamento da Yolo



Atividade

- Junta-se com o seu grupo, escolha 100 imagens do conjunto do seu trabalho e rotule-as.
- Em seguida faça um treinamento com a Yolo de um detector de objetos para o seu trabalho final